# [COLLEGE / UNIVERSITY NAME PLACEHOLDER]

**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING / INFORMATION TECHNOLOGY** B.Tech Second Year | Course: Internet Programming Laboratory [Subject Code: _____]

## ASSIGNMENT REPORT: CINEBOOK – MOVIE TICKET BOOKING WEBSITE

**Academic Session:** 2026–2027 **Faculty Guide:** [Faculty Name / Designation] **Project Type:** Full-Stack Group Web Application (5 Members) **Submission Date:** November 2026

### 1. Introduction & Objectives

CineBook is a full-stack, responsive movie ticket booking web application developed to satisfy the curriculum objectives of the B.Tech Internet Programming syllabus. The application bridges client-side interactive DOM interfaces with an asynchronous Node.js/Express REST backend and a relational MySQL database. The primary goals are: (a) enabling users to securely register, authenticate, and reserve cinema tickets with interactive seat selection; (b) validating transaction data and preventing double-booking; and (c) providing an administrative CRUD module to manage cinema catalogs.

### 2. Technologies Used

● **Frontend:** HTML5 Semantic Elements, CSS3 (Flexbox, Grid, Variables, @media print), Vanilla JavaScript (ES6+), DOM Manipulation, Fetch API. ● **Backend:** Node.js runtime, Express.js REST server, express-session (cookie session state), bcryptjs (password hashing), dotenv. ● **Database:** MySQL relational database, mysql2 promise pool, parameterized SQL queries preventing SQL Injection.

### 3. Explanation of the Five Connected Webpages & Workflow

● **Workflow:** Movie Selection → Theater Selection → Date & Showtime → Seat Matrix → Confirmation Ticket. **1. Page 1 – Registration (signup.html):** Validates full name, email regex, password strength, and confirmation equality. Stores accounts using salted bcrypt hashes on the server to prevent duplicate email registrations. **2. Page 2 – Login (login.html):** Authenticates credentials against stored password hashes. Establishes express-session cookie state and redirects users back to their pending booking destination. **3. Page 3 – Movie Browsing (movies.html):** Dynamically queries GET /api/movies and constructs DOM cards with titles, genres, durations, and pricing. Features live client-side search and genre filtering without page reloads. **4. Page 4 – Theater & Seat Booking (booking.html):** Allows selecting cinema theaters dynamically (IMAX Laser, PVR Premiere, Cinepolis 4DX, Royale VIP), show date, showtime, and interactive 5x8 seat grid. Checks theater-specific seat occupancy, prevents duplicate bookings in the same hall, and recalculates totals on the server. **5. Page 5 – Ticket Confirmation & Printing (confirmation.html):** Fetches verified booking details from GET /api/booking/:id with ownership protection. Displays cinema ticket with theater name, screen, location, barcode, and triggers browser printing via JavaScript window.print() formatted with @media print rules. **6. Module IV Bonus – Admin CRUD (admin.html):** Role-protected dashboard allowing administrators to Create, Read, Update, and Delete movies and monitor bookings.

### 4. MySQL Database Schema Explanation

The relational database "cinebook" is structured into four normalized tables with foreign key constraints: ● **users table:** user_id (INT, PK, AI), name (VARCHAR), email (VARCHAR, UNIQUE), password (VARCHAR, bcrypt hash), role (VARCHAR, default "user"). ● **movies table:** movie_id (INT, PK, AI), title (VARCHAR), description (TEXT), genre (VARCHAR), duration (INT min), price (DECIMAL), poster (VARCHAR). ● **theaters table:** theater_id (INT, PK, AI), theater_name (VARCHAR), location (VARCHAR), screen_name (VARCHAR). ● **bookings table:** booking_id (INT, PK, AI), user_id (FK to users), movie_id (FK to movies), theater_id (FK to theaters), show_date (DATE), show_time (VARCHAR), seats (VARCHAR), ticket_count (INT), total_price (DECIMAL), created_at (TIMESTAMP).

### 5. Individual Contributions of All Five Group Members

| Member | Student Name / Roll | Assigned Page | Implemented Features & Syllabus Concepts |
|---|---|---|---|
| **Member 1** | [Student 1 Name] [Roll Number 1] | Page 1: Registration (signup.html) | Form validation (regex, length, equality), Fetch POST /api/signup, bcrypt hashing, error alert handling, redirect. |
| **Member 2** | [Student 2 Name] [Roll Number 2] | Page 2: Login (login.html) | Authentication via POST /api/login, express-session state, HTTP-only cookie, redirect logic, feedback banners. |
| **Member 3** | [Student 3 Name] [Roll Number 3] | Page 3: Movie Catalog (movies.html) | Dynamic card DOM creation from GET /api/movies, client search and genre filtering, responsive grid styling. |
| **Member 4** | [Student 4 Name] [Roll Number 4] | Page 4: Seat Booking (booking.html) | Interactive 5x8 seat map, booked seat detection, client-side dynamic pricing, login check guard, server revalidation. |
| **Member 5** | [Student 5 Name] [Roll Number 5] | Page 5: Ticket Output (confirmation.html) | Booking retrieval via GET /api/booking/:id, ticket layout, window.print() integration, @media print formatting. |

### 6. Conclusion

The CineBook project successfully fulfills all curriculum requirements for the B.Tech Internet Programming laboratory coursework. By combining clean semantic HTML5, modern CSS3 styling, vanilla JavaScript DOM interactivity, Node.js/Express

REST APIs, and a relational MySQL database, the project demonstrates real-world software engineering practices. All 16 automated backend test assertions passed, guaranteeing security, price integrity, and double-booking prevention. The codebase is modular, clean, and easily explainable in an academic viva.

Student Signatures: (1) _____ (2) _____ (3) _____ (4) _____ (5) _____ Faculty Examiner Signature: _____ Date: _____